

LABORATOR 4
Ana-Maria Corcodel
GRUPA 10 216

Main modificat.c:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define NO_THREADS 4 // numărul de thread-uri folosite în execuție

// Funcție care scrie într-un fișier tipul metodei, timpul și suma obținută
void write_file(char *filename, char *type, double time, long sum) {
    FILE *file = fopen(filename, "w+");
    if (file == NULL) {
        perror("fopen");
        return;
    }
    fprintf(file, "Type: %s\nTime: %lf\nSum: %ld", type, time, sum);
    fclose(file);
}

int main() {
    // deschidem fișierul de intrare
    FILE *file = fopen("input.txt", "r");

    int size = 0, i;
    int *numbers = NULL;

    // cele 3 sume pentru cele 3 metode:
    // reduction, atomic și critical
    long sum1 = 0, sum2 = 0, sum3 = 0;

    // t1 va reține momentul de start al măsurării
```

```

double t1 = 0.0;

// timpii pentru cele 3 variante
double time1 = 0.0, time2 = 0.0, time3 = 0.0;

// verificăm dacă fișierul a fost deschis corect
if (file == NULL) {
    perror("input.txt");
    return 1;
}

// setăm explicit numărul de thread-uri
omp_set_num_threads(NO_THREADS);

// regiune paralelă principală
// variabilele din shared sunt comune tuturor thread-urilor
#pragma omp parallel shared(file, size, numbers, sum1, sum2, sum3, time1, time2, time3, t1)
{
    // un singur thread citește datele din fișier
    // evităm citirea multiplă a acelorași date
    #pragma omp single
    {
        // prima valoare din fișier reprezintă dimensiunea vectorului
        fscanf(file, "%d", &size);

        // alocăm memorie pentru vector
        numbers = (int *)malloc(size * sizeof(int));
        if (numbers == NULL) {
            perror("malloc");
            fclose(file);
            exit(1);
        }

        // citim toate numerele din fișier
        for (int j = 0; j < size; j++) {
            fscanf(file, "%d", &numbers[j]);
        }

        // închidem fișierul după citire
        fclose(file);
    }
}

```

```

}

// toate thread-urile așteaptă până când citirea este completă
#pragma omp barrier

// ----- VARIANTA 1: REDUCTION -----

// un singur thread inițializează suma și pornește cronometrul
#pragma omp master
{
    sum1 = 0;
    t1 = omp_get_wtime();
}

// ne asigurăm că toate thread-urile pornesc după inițializare
#pragma omp barrier

// împărțim iterațiile buclei între thread-uri
// reduction creează o copie locală a sumei pentru fiecare thread
// iar la final toate valorile sunt combinate automat
#pragma omp for reduction(+:sum1)
for (i = 0; i < size; i++) {
    sum1 += numbers[i];
}

// așteptăm terminarea tuturor thread-urilor
#pragma omp barrier

// thread-ul master oprește cronometrul
#pragma omp master
{
    time1 = omp_get_wtime() - t1;
}

#pragma omp barrier

// ----- VARIANTA 2: ATOMIC -----

// inițializare și pornire cronometru
#pragma omp master

```

```

{
    sum2 = 0;
    t1 = omp_get_wtime();
}

#pragma omp barrier

// fiecare thread parcurge o parte din vector
// atomic protejează operația simplă de adunare
#pragma omp for
for (i = 0; i < size; i++) {
    #pragma omp atomic
    sum2 += numbers[i];
}

#pragma omp barrier

// salvăm timpul total pentru atomic
#pragma omp master
{
    time2 = omp_get_wtime() - t1;
}

#pragma omp barrier

// ----- VARIANTA 3: CRITICAL -----

// inițializare și pornire cronometru
#pragma omp master
{
    sum3 = 0;
    t1 = omp_get_wtime();
}

#pragma omp barrier

// fiecare thread execută o parte din buclă
// dar actualizarea sumei se face într-o zonă critical
// unde intră un singur thread la un moment dat
#pragma omp for

```

```

for (i = 0; i < size; i++) {
    #pragma omp critical
    {
        sum3 += numbers[i];
    }
}

#pragma omp barrier

// salvăm timpul pentru varianta critical
#pragma omp master
{
    time3 = omp_get_wtime() - t1;
}

#pragma omp barrier

// ----- SCRIERE ÎN FIȘIERE CU SECTIONS -----

// sections permite executarea mai multor task-uri diferite în paralel
// aici fiecare secțiune scrie într-un alt fișier
#pragma omp sections
{
    #pragma omp section
    write_file("out_reduction.txt", "reduction", time1, sum1);

    #pragma omp section
    write_file("out_atomic.txt", "atomic", time2, sum2);

    #pragma omp section
    write_file("out_critical.txt", "critical", time3, sum3);
}

// eliberăm memoria alocată pentru vector
free(numbers);
return 0;
}

```

tree modificat.c:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define NO_THREADS 8 // numărul de thread-uri pentru calculul pe arbore

// definirea structurii unui nod din arborele binar de căutare
typedef struct binary_search_tree {
    int data;
    struct binary_search_tree *left;
    struct binary_search_tree *right;
} binary_search_tree;

// funcție de inserare într-un arbore binar de căutare
binary_search_tree *insert(binary_search_tree *tree, int data) {
    // dacă arborele este gol, creăm un nod nou
    if (tree == NULL) {
        binary_search_tree *new_node = malloc(sizeof(binary_search_tree));
        new_node->data = data;
        new_node->left = NULL;
        new_node->right = NULL;
        return new_node;
    }
    // dacă valoarea este mai mică sau egală, mergem în stânga
    else if (tree->data >= data) {
        tree->left = insert(tree->left, data);
    }
    // altfel mergem în dreapta
    else {
        tree->right = insert(tree->right, data);
    }
    return tree;
}

// parcurgere de tip preorder: rădăcină, stânga, dreapta
void preorder(binary_search_tree *tree) {
    if (tree == NULL) {
        return;
    }
}
```

```

    }

    printf("%d ", tree->data);
    preorder(tree->left);
    preorder(tree->right);
}

// funcție ajutătoare care întoarce maximul dintre două valori
int maximum(int a, int b) {
    return a >= b ? a : b;
}

// calculează înălțimea arborelui folosind task-uri OpenMP
int height(binary_search_tree *tree) {
    // dacă nodul este null, înălțimea este 0
    if (tree == NULL) {
        return 0;
    }

    int left = 0, right = 0;

    // creăm un task pentru calculul înălțimii subarborelui stâng
    #pragma omp task shared(left)
    {
        left = height(tree->left);
    }

    // creăm un task pentru calculul înălțimii subarborelui drept
    #pragma omp task shared(right)
    {
        right = height(tree->right);
    }

    // așteptăm ca ambele task-uri să se termine
    #pragma omp taskwait

    // înălțimea arborelui curent este
    // 1 + maximul dintre înălțimea din stânga și cea din dreapta
    return maximum(left, right) + 1;
}

```

```

int main() {
    // setăm numărul de thread-uri
    omp_set_num_threads(NO_THREADS);

    // construim arborele manual prin inserare
    binary_search_tree *tree = NULL;
    tree = insert(tree, 8);
    tree = insert(tree, 3);
    tree = insert(tree, 10);
    tree = insert(tree, 1);
    tree = insert(tree, 6);
    tree = insert(tree, 14);
    tree = insert(tree, 4);
    tree = insert(tree, 7);
    tree = insert(tree, 13);

    // afișăm parcurgerea preorder a arborelui
    preorder(tree);
    printf("\n");

    // pornim regiunea paralelă
    #pragma omp parallel
    {
        // un singur thread pornește apelul inițial al funcției height
        // de aici se creează task-urile recursive
        #pragma omp single
        {
            printf("Height = %d\n", height(tree));
        }
    }
    return 0;
}

```


}

```
vboxuser@Ubuntu:~/Desktop$ gcc main_modificat.c -o main_modificat -O2 -fopenmp -lm
vboxuser@Ubuntu:~/Desktop$ gcc tree_modificat.c -o tree_modificat -O2 -fopenmp -lm
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=1
vboxuser@Ubuntu:~/Desktop$ ./main_modificat
vboxuser@Ubuntu:~/Desktop$ cat out_reduction.txt
Type: reduction
Time: 0.000004
Sum: 4992041vboxuser@Ubuntu:~/Desktop$ cat out_atomic.txt
Type: atomic
Time: 0.000099
Sum: 4992041vboxuser@Ubuntu:~/Desktop$ cat out_critical.txt
Type: critical
Time: 0.000892
Sum: 4992041vboxuser@Ubuntu:~/Desktop$
vboxuser@Ubuntu:~/Desktop$ export OMP_NUM_THREADS=4
vboxuser@Ubuntu:~/Desktop$ ./main_modificat
vboxuser@Ubuntu:~/Desktop$ cat out_reduction.txt
Type: reduction
Time: 0.000004
Sum: 4992041vboxuser@Ubuntu:~/Desktop$ cat out_atomic.txt
Type: atomic
Time: 0.000180
Sum: 4992041vboxuser@Ubuntu:~/Desktop$ cat out_critical.txt
Type: critical
Time: 0.001085
Sum: 4992041vboxuser@Ubuntu:~/Desktop$ ./tree_modificat
8 3 1 6 4 7 10 14 13
Height = 4
vboxuser@Ubuntu:~/Desktop$
```

Metodă	1 thread (s)	4 thread-uri (s)
reduction	0.000004	0.000004
atomic	0.000099	0.000180
critical	0.000892	0.001085

Observații:

- reduction este **cea mai rapidă metodă**
- atomic este mai lentă decât reduction, dar mai rapidă decât critical
- critical este **cea mai lentă**, deoarece introduce blocare (lock) între thread-uri